Minicurso
Lógica de programação

Minicurso Lógica com Python

Material Didático — Semana da Computação C3

Jhon Victor Ramos Martins

Redes Profissionais	Contato
LinkedIn: jhon-victor-ramos	**Instagram:** @jhonvictor_dev
GitHub: Jhon-Victor-Ramos	**E-mail:** jhon.vrmartins@gmail.com

Sumário

1 Parte I: Os Fundamentos **2**

1.1 O que é a Lógica de Programação? 2

1.2 O Fluxograma 2

2 Parte II: Estruturas Básicas **2**

2.1 Variáveis e Entradas 2

2.2 Indentação 3

2.3 Estruturas Condicionais 3

2.4 Estruturas de Repetição 5

3 Funções e Métodos **8**

3.1 O que são os parâmetros? 9

3.2 Operadores Lógicos 9

Bônus: Boas Práticas **10**

1 Parte I: Os Fundamentos

1.1 O que é a Lógica de Programação?

Programar é, essencialmente, **dar ordens claras e em sequência** para uma máquina extremamente rápida, mas que **não faz literalmente nada além** do que se pede.

Academicamente, a lógica de programação é o uso correto das *leis do pensamento* e de processos de raciocínio para a produção de soluções coerentes na programação de computadores. Ou seja, é a capacidade de encadear pensamentos e passos lógicos para atingir um objetivo definido previamente.

A programação é, portanto, a construção de **algoritmos** — que são, de modo formal, uma sequência finita de passos que levam uma tarefa a ser executada, como um manual para montar um quebra-cabeça para o computador.

1.2 O Fluxograma

Antes de escrevermos um programa usando código (como o Python, linguagem utilizada nesse minicurso), precisamos desenhar a lógica de uma forma que o *nosso* cérebro entenda. Para isso usamos o **Fluxograma**, que é a esquematização visual do nosso programa.

Ele utiliza uma regra de elementos com formas geométricas para representar as ações. São eles:

- ○ **Ovais:** Indicam o `Início` e o `Fim` da aplicação.
- □ **Paralelogramos:** Representam a **Entrada** (o usuário digitando algo, por exemplo) e a **Saída** (o computador exibindo mensagens na tela). *(Adaptado para ícone retangular genérico)*
- ▭ **Retângulos:** É o processamento. Onde ocorrem cálculos matemáticos ou informações são armazenadas.
- ◊ **Losangos:** Representa as condicionais. Quando o programa chega aqui, alguma pergunta que só aceita duas respostas (*Verdadeiro* ou *Falso*) é feita. Dependendo da resposta, o rumo do programa é alterado.
- ↔ **As Setas:** A ação dos elementos com relação ao tempo.

2 Parte II: Estruturas Básicas

2.1 Variáveis e Entradas

Variáveis são espaços alocados na memória do computador para armazenar dados **temporariamente** durante a execução de um programa. O valor contido nesse espaço pode ser alterado (ou seja, variar) ao longo do tempo.

Para que o programa seja interativo e receba dados fornecidos pelo usuário, utiliza-se a função de entrada `input()`. Para exibir os resultados e dados armazenados no terminal, a função de saída `print()` é a usada.

```
print("Bem-vindo!")
nome = input("Qual o seu nome? ")
idade = int(input("Qual a sua idade? "))
```

Dica: O `int()` "abraçando"o `input` transforma o texto digitado em um número inteiro, permitindo que a gente faça contas com a idade depois, se necessário.

2.2 Indentação

A indentação refere-se ao recuo do texto em relação à margem esquerda da página, geralmente é aplicado usando a tecla `Tab` (ou 4 espaços). Enquanto em muitas outras linguagens de programação o recuo é uma prática utilizada apenas para facilitar a leitura humana, em Python é uma **exigência**.

O computador utiliza esse alinhamento para diferenciar a hierarquia das instruções, determinando quais linhas de código estão dentro de um determinado bloco lógico. Se houver um recuo incorreto ou inconsistente, o programa deve apresentar um erro de sintaxe chamado `IndentationError`, impedindo execução do mesmo. Portanto, criar algoritmos que funcionem, nessa linguagem, exige o bom uso da indentação.

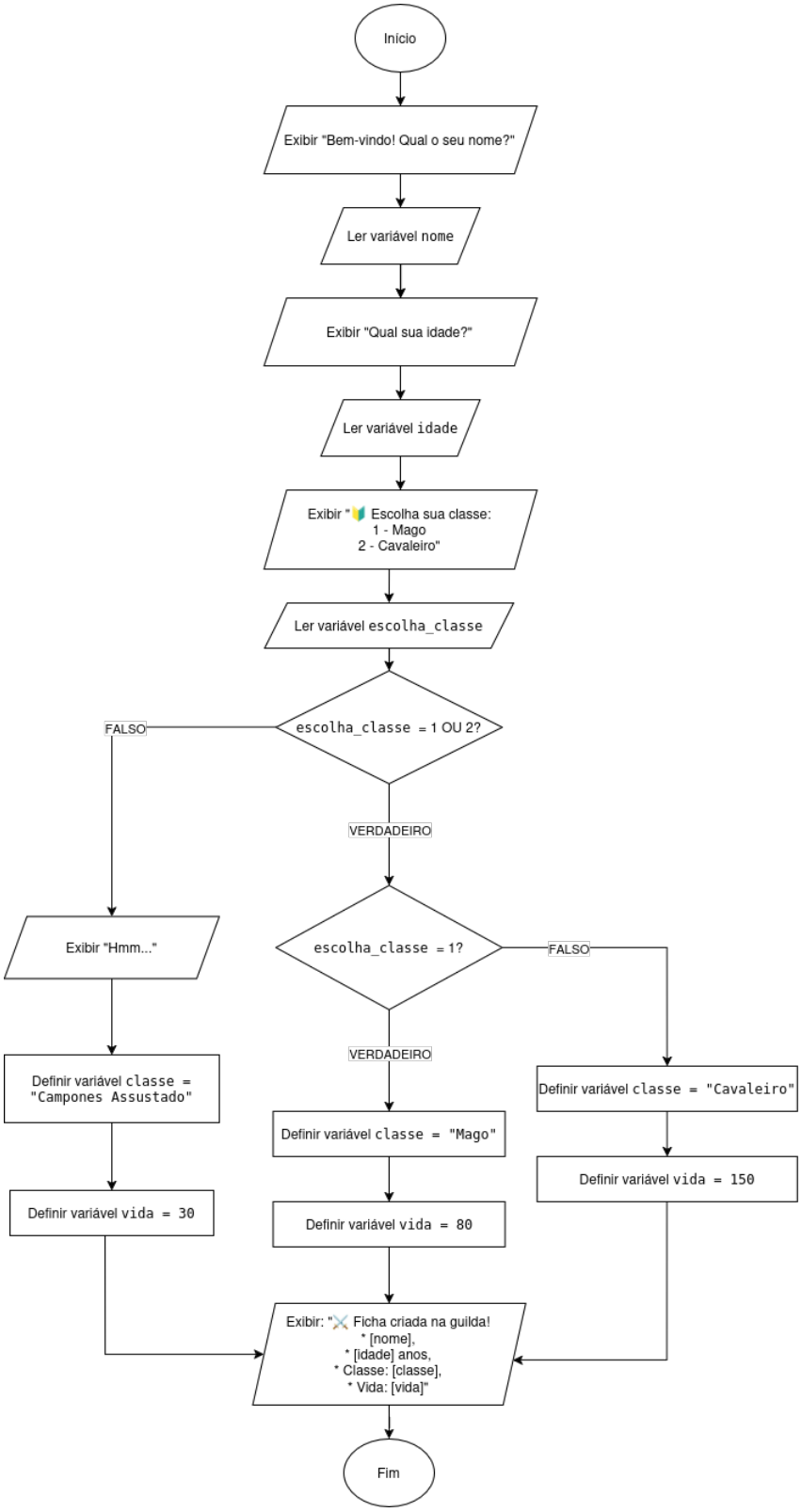
Dica:

Observe que nos blocos de código, nas futuras seções desse material, possuem um espaço em relação à esquerda. Esse espaço é a aplicação da indentação na prática.

2.3 Estruturas Condicionais

As estruturas condicionais (chamadas também de controle de fluxo) são utilizadas para desviar a execução linear do código com base em avaliações lógicas. A estrutura básica em Python utiliza `if` (se), `elif` (abreviação de *else if* — senão se) e `else` (senão). O bloco de código posto dentro desses escopos só será executado caso a condição testada retorne um valor booleano verdadeiro (`True`) para a condição imposta.

No contexto do projeto idealizado para esse minicurso, as condicionais, inicialmente, serão utilizadas para definir os atributos do personagem com base na escolha do usuário. A lógica está representada no fluxograma abaixo:



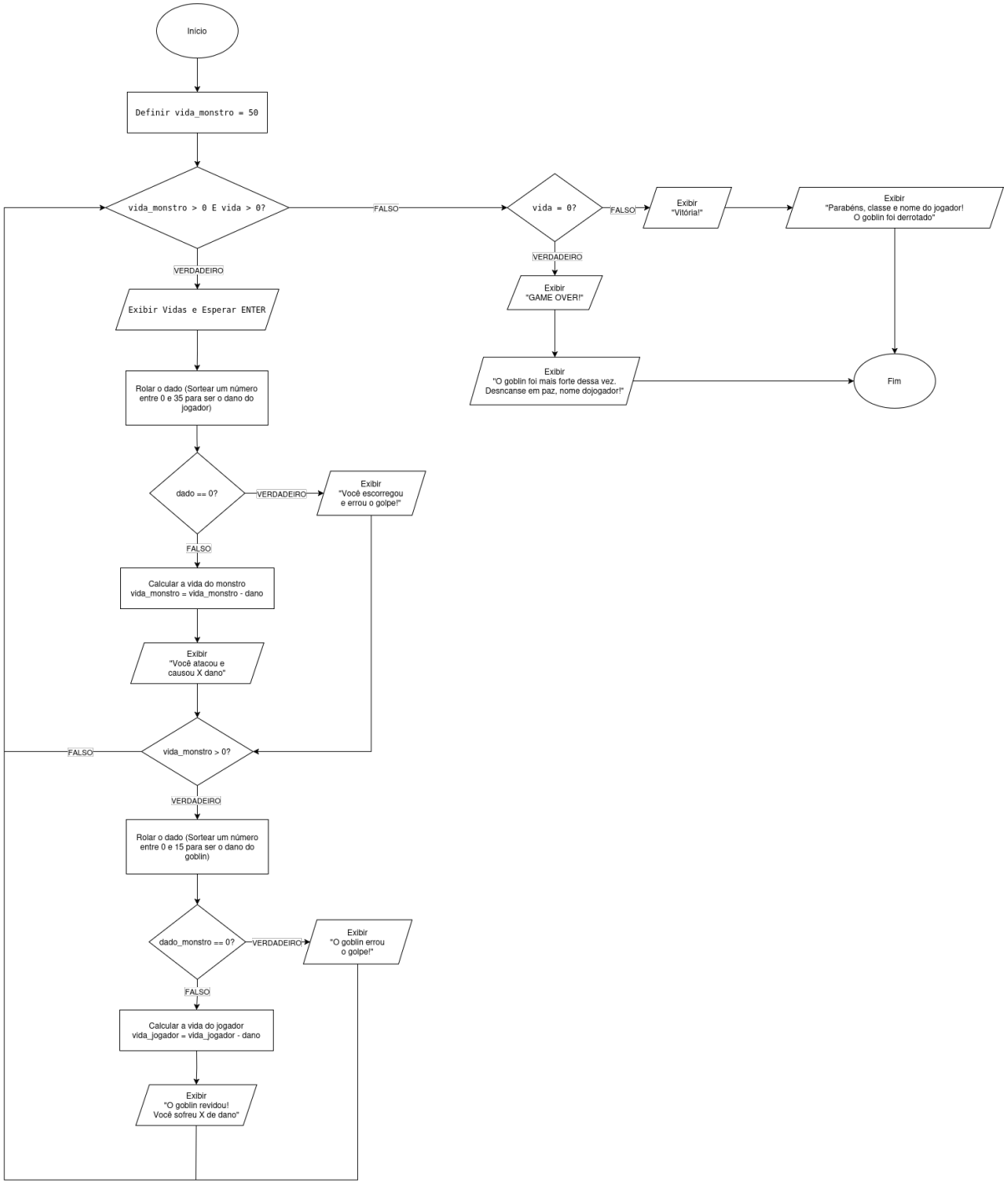
Veja a implementação do fluxograma (tratando das partes de decisões, já que a inicial foi tratada anteriormente) acima em código *Python*:

```python
escolha_classe = int(input("""
  Escolha sua classe:
    1 - Mago
    2 - Cavaleiro
Resposta: """))

if escolha_classe == 1 or escolha_classe == 2:
    if escolha_classe == 1:
        classe = "Mago"
        vida = 80
    else:
        classe = "Cavaleiro"
        vida = 150
else:
    print("Hmm...")
    classe = "Campones Assustado"
    vida = 30

print(f"""
  Ficha criada na guilda!
    * {nome}, {idade} anos,
    * Classe: {classe}
    * Vida: {vida}""")
```

2.4 Estruturas de Repetição

Por vezes, faz-se necessário que um bloco de código seja executado múltiplas vezes. As estruturas de repetição, também conhecidas como laços ou *loops*, são as responsáveis por isso. O laço `while` (enquanto) verifica uma expressão lógica antes de cada iteração, repetindo as instruções contidas em seu escopo enquanto a condição posta permanecer verdadeira.

Para complementar o desenvolvimento do projeto prático, é necessário introduzir o conceito de **Bibliotecas** (ou módulos). Tratam-se de conjuntos de códigos e funções já escritas que podem ser importados para o arquivo atual. Neste exemplo, utilizaremos a biblioteca embutida `random` para a geração de números pseudoaleatórios.



Observe a implementação do fluxograma acima em linguagem *Python*:

```python
import random

print("\n" + "="*30)
print(" UM INIMIGO APARECEU! ")
print("Um Goblin raivoso bloqueia a porta da guilda!")
vida_monstro = 50

# O laco continua verdadeiro SE o monstro E o jogador possuirem
   vida acima de 0
while vida_monstro > 0 and vida > 0:
    print(f"\n Vida do Goblin: {vida_monstro} |  Sua Vida: {vida}")
    input("Pressione ENTER para rolar os dados e atacar! ")

    # Processamento do dano do jogador
    dano_jogador = random.randint(0, 35)

    if dano_jogador == 0:
        print(" Voce escorregou e ERROU O ATAQUE!")
    else:
        vida_monstro = vida_monstro - dano_jogador
        print(f" Voce acertou em cheio e causou {dano_jogador} de
           dano!")

    # Processamento do dano do monstro
    if vida_monstro > 0:
        dano_goblin = random.randint(0, 15)

        if dano_goblin == 0:
            print(" O goblin errou o golpe!")
        else:
            vida = vida - dano_goblin
            print(f" O Goblin revidou! Voce sofreu {dano_goblin} de
                dano.")

print("\n" + "="*30)

# Verificacao do estado final das variaveis para determinar a saida
if vida > 0:
    print(" VITORIA!")
    print(f"Parabens, {classe} {nome}! O Goblin foi derrotado.")
else:
    print(" GAME OVER...")
    print(f"O Goblin foi mais forte dessa vez. Descanse em paz, {
       nome}.")
```

3 Funções e Métodos

As funções e métodos referem-se às estruturas de código que agrupam um conjunto de instruções para realizar, idealmente, **uma** tarefa específica. Ao invés de escrevermos um mesmo trecho diversas vezes, podemos separar essa lógica em uma função ou método e nomeamos esse bloco. Dessa forma, o código fica mais limpo e enxuto.

Embora, na prática, as duas estruturas sejam parecidas, existe uma diferença importante:

- **Função:** São blocos genéricos criados independentemente que fazem uma ação. No códigos, chamamos a função e passamos **onde** ela irá atuar
  - São exemplos de `funções`:
    - `calcularMedia(nota1, nota2)`
    - `cadastrarAluno(nome, idade, turma, curso)`
- **Método:** São funções que já nascem amarradas a um objeto específico, como uma lista ou uma string, por exemplo. O próprio objeto sabe como executar essa ação sobre ele mesmo.
  - São exemplos de `Métodos`:
    - `.upper()` ← Deixa a string antes do código em letras maiúsculas.
    - `.lower()` ← Deixa a string antes do código em letras minúsculas.

Atenção!

Um erro comum no início é confundir a impressão de algo na tela — `print()` — com o retorno dessa informação no código — `return`. O `print()` apenas exibe a informação na tela para o usuário ler, mas o programa não guarda esse dado em seguida. Já o `return` devolve o resultado para o código, permitindo que ele seja salvo em uma variável e utilizado em momentos posteriores!

Estrutura de uma função em Python Para criar uma função, utilizamos uma palavra que a própria linguagem já deixa separada para isso — essas palavras recebem o nome de "palavra **reservada**"— sendo ela: `def` (uma abreviação para *define*). A estrutura básica de uma função é a seguinte:

```
def nome_da_funcao(parametros):
    # Repare na identaćão
    codigo

    return resultado # Aparece somente se for necessário um retorno
```

3.1 O que são os parâmetros?

Um dos conceitos que mais causa confusão no início é entender que o nome do parâmetro dentro da função **não tem nenhuma relação** com o nome da variável que passamos para ela do lado de fora. O parâmetro na definição da função poderia, por exemplo, se chamar `jujuba`, e o código interno da função usaria `jujuba`, mesmo que uma variável chamada `mediaAluno` fosse, na chamada da função, enviada. Em uma analogia, podemos pensar na função como um liquidifacador que. Para que seja feito um suco é necessário que, no liquidificador, os ingredientes sejam postos. Esses ingredientes, na programação, são os parâmetros.

Eles — os parâmetros — são, portanto, **espaços reservados** que a função cria para receber dados.

A confusão acontece pois o **parâmetro é só um apelido interno** da função. O nome da variável que é utilizada no código **não importa** para a função. Veja o seguinte exemplo:

```
def fazer_suco(fruta, liquido):
    print(f"Batendo {fruta} com {liquido}!")

# Caso 1: Nomes diferentes
minha_maca = "Maćã"
meu_leite = "Leite"
fazer_suco(minha_maca, meu_leite)
# A funćão recebe "Maćã" e apelida internamente de 'fruta'.

# Caso 2: Nomes iguais
fruta = "Morango"
liquido = "Água"
fazer_suco(fruta, liquido)
```

No segundo caso parece que a variável `fruta` e o parâmetro `fruta` são a mesma coisa. Mas **não** são! São como duas pessoas com o mesmo nome mas que vivem em cidades diferentes. Embora textualmente seja igual, as entidades não são.

Por conta das boas práticas de programação, pode ser que as variáveis e os parâmetros acabem tendo nomes iguais pois facilita a leitura do código (por exemplo: *"A variável fruta é passada para o espaço da fruta na função"*). Contudo, vale ressaltar que a função **só se importa com o conteúdo** do que é passado, não o nome que aquilo possui.

3.2 Operadores Lógicos

Por vezes, faz-se necessário testar mais de uma condição ao mesmo tempo para tomar uma decisão. Para isso, utilizamos os **operadores lógicos**: `and`, `or` e `not`. Eles funcionam exatamente como no dia a dia.

- `and` **(E): Tudo** precisa ser verdade.
  - *Exemplo:* Para entrar no cinema, você precisa ter o ingresso **E** (`and`) estar no horário. Se um dos dois for falso, você não entra.

  - `if vida > 0 and monstro_vivo == True:` (O jogo só continua se as duas coisas forem verdadeiras).

- `or` **(OU): Pelo menos um** precisa ser verdade.
  - *Exemplo:* Para ganhar a promoção, você precisa comprar o produto A **OU** (`or`) o produto B. Qualquer um dos dois serve.
  - `if classe == "Mago"or classe == "Guerreiro":` (Aceita qualquer uma das duas opções).

- `not` **(NÃO):** Inverte a lógica. O que era verdade vira mentira, e vice-versa.
  - `if not vida == 0:` (Lê-se: "Se a vida NÃO for igual a zero").

Bônus: Boas Práticas

Escrever um código que a máquina entenda é o requisito mínimo. Agora, escrever um código que **outros seres humanos** (e você mesmo, em momentos posteriores) entendam faz parte de um bom desenvolvedor.

A comunidade Python segue um guia oficial de estilo chamado **PEP 8**, que fala as melhores práticas para manter os códigos limpos, legíveis e padronizados. Aqui estão três regras para você aplicar desde o seu primeiro projeto:

1. **Nomenclatura Clara e Explícita:** Nunca deixe faltar letras se isso custar a clareza do programa.
   - Exemplo:
     - Ruim: `v = 50` ou `a = b + 10`
     - Bom: `vida_jogador = 50` ou `valor_atual = precoAntigo + 10`
2. **O Padrão `snake_case`: Em Python**, o padrão para nomear variáveis é utilizar letras minúsculas e separar as palavras com um *underline* (sublinhado), o nome é `snake_case` pois essa escrita acaba imitando o rastro de uma cobra.
   - Exemplo:
     - Ruim: `EscolhaClasse` ou `escolhaClasse`
     - Bom: `escolha_classe`
3. **Respiro:** Utilize um espaço em branco antes e depois de operadores de atribuição (`=`) e operadores matemáticos (`+`, `-`, `>`, `<`). Isso facilita a leitura das expressões.
   - Exemplo:
     - Ruim: `vida=5+20` ou `if vida<=0:`
     - Bom: `vida = 5 + 20` ou `if vida <= 0:`